NPRT630 | PHASE 4

Final Project Report

SyncStock 2.0
SME inventory and access management system

Implementation review against the project documents, submitted source code, automated checks and the AWS-hosted live system.

Overall finding

The project delivers a substantial, AWS-hosted inventory prototype. During this review, both web portals responded, the live API reported healthy, and its readiness endpoint confirmed PostgreSQL storage. Backend tests and production builds also passed. The hosted demonstration is HTTP-only and single-instance; a successful live sign-in and end-to-end business transaction were not verified.

IMPLEMENTED IN PROTOTYPE | PARTIALLY IMPLEMENTED | NOT DELIVERED / NOT VERIFIED

Assessment date: 12 October 2026
Prepared from the supplied NPRT630 project materials.

01 | EXECUTIVE ASSESSMENT

What this report establishes

SyncStock addresses the documented SME retail problem with a role-based inventory application for core stock, sales, receiving and replenishment. The source and running AWS demo establish a deployed modular prototype, not the distributed, cloud-scale platform described in the proposal.

> **Phase 4 technology-stack variance:** the lecturer's briefing prescribes Java and Oracle or SQL database connectivity. The submitted application uses TypeScript/Node.js, React and PostgreSQL. It uses a relational SQL database but does not meet the prescribed Java-language stack.

Assessment area	Finding
Core prototype functionality	**Implemented in code** Worker and admin portals, role-protected APIs, stock management and principal sales/warehouse/procurement flows are present.
AWS hosting	**Confirmed** The roadmap identifies a single EC2 demo in Europe (Stockholm). The AWS worker/admin pages and API responded during this review.
Live database readiness	**Confirmed** The live API readiness response identified PostgreSQL storage and a ready status.
Backend tests and builds	**Passed** This review: 76 backend tests passed; backend TypeScript and frontend production builds succeeded.
Live authenticated workflow	**Not verified** No credentials were used; a role-specific sign-in, stock mutation and persisted transaction were not demonstrated in this review.
Proposed distributed architecture	**Not delivered** The running application is one Express API with in-process modules and one PostgreSQL database, not independently deployed services.

HTTP security caution: the AWS demo does not have HTTPS enabled. Do not enter real passwords or sensitive business data into the hosted site.

How completion is judged

Implemented in code means the source, screens or automated tests contain the capability; it does not imply every feature was exercised live. **Partially implemented** means a prototype exists but a documented integration, requirement or operational target is incomplete. **Not verified live** means this assessment did not observe the workflow operating end to end.

Earlier phases versus the submitted system

The problem statement and development plan define a cloud-native SME retail solution. The system design specification details role-based screens, data and processes. The Phase 4 briefing asks for implementation, live demonstration, testing and a consolidated final report using its prescribed technical stack.

Documented requirement or design	What the implementation provides	Assessment
Products, stock locations, sales and returns	Product/barcode lookup, warehouse/bin stock, reservations and movements, cashier checkout, receipts and receipt-based returns.	Code implemented
Receiving, cycle counts and reconciliation	Receiving and partial PO receipt, bin counts and stock adjustments. Damage notes are not fully sent by the UI; cycle-count approval is absent.	Partial
Replenishment and suppliers	Supplier records, low-stock draft POs, approval, local sent status, partial/full receipt and reviews. "Sent" does not notify or transmit to the vendor.	Partial
Forecasting and reporting	Simple regression and sparse-data fallback, reorder/stockout-risk calculations, operational KPIs. No validated accuracy, configured demand feed, EOQ or full inventory-turnover report.	Partial
Role-specific user experiences	Worker and admin portals, five roles and route authorization. Some real route permissions differ from the target access matrix.	Implemented with variance
Cloud microservices and event-driven integration	One Express API, in-process domain modules and one PostgreSQL database. Provider-neutral adapter contracts are present; no active vendor adapters.	Not delivered
Phase 4 stack and demonstration	AWS-hosted app pages and PostgreSQL readiness are live. Runtime is TypeScript/Node.js, not prescribed Java.	Demonstrated with variance

Target versus result: figures in the academic proposals—including reliability, recovery, performance and forecast-improvement targets—are not measured results unless supported by actual test evidence.

03 | COMPLETED IN THE PROTOTYPE

What has been implemented

Inventory and store operations

- Product catalogue with SKU/barcode lookup, supplier references, pricing and reorder settings.
- Warehouse/bin inventory, reserved versus available stock, movements and low-stock signals.
- Cashier flows for discount, VAT calculation, cash tender/change, receipt creation and returns.
- Warehouse flows for receiving, partial PO receipts, cycle counts, adjustments and order stock reserve/commit/release.

Procurement and decision support

- Supplier creation, archive/restore and reference-protected deletion.
- Draft low-stock POs, approval, cancellation, sent status and partial/full receiving.
- Supplier-review scores, dashboard KPIs and reorder/stockout-risk views.
- Linear-regression prototype based on sales movements, with sparse-history fallback.

Administration and security foundations

- Separate worker operations and admin governance portal entry points.
- ADMIN, MANAGER, CASHIER, WAREHOUSE_STAFF and PROCUREMENT_STAFF roles with API route authorization.
- Employee, warehouse, company-profile and policy administration; application audit events.
- Hashed password support, first-login setup, cookie sessions, CSRF checks, rate limits and security headers.
- TOTP enrolment/verification, email verification and production bot-protection/HTTPS checks are represented; some rely on deployment configuration.

Engineering evidence

- 76 backend automated tests passed in this review with zero failures.
- Backend TypeScript compilation and frontend multi-page production build succeeded.
- AWS production readiness reported `ready` with `postgres` storage.

> **Evidence boundary:** AWS page and readiness checks show an accessible host and database readiness, not that every UI workflow, authentication policy, notification or financial transaction has been tested on the hosted database.

What is incomplete or not verified

Partially implemented

- **AWS deployment/operations:** one EC2 host serves the application with PostgreSQL and health/readiness checks. HTTPS, high availability, tested backup/restore, CI/CD, centralized monitoring and demonstrated service-level objectives are absent.
- **Live workflows:** both browser portals load and API/PostgreSQL readiness passes, but this assessment did not sign in or perform an authenticated sale, stock change or purchase-order transaction.
- **Electronic payments/refunds:** provider interface and fail-closed handling exist; no payment provider is configured by default.
- **Forecasting:** regression/reorder prototype exists, not the proposal's validated ML service or configured seasonal/promotion feed and EOQ optimization.
- **Security/compliance:** application controls exist, but TOTP is not required at every login. Email, bot protection, TLS/HTTPS and database TLS depend on configuration. Audit immutability is not enforced by the database.
- **Receiving and fulfillment:** condition/damage notes are not completely sent from the UI; online order stock actions are local, with no connected commerce platform or picking-task workflow.
- **Supplier communication:** POs can be marked sent but are not emailed or transmitted by EDI/vendor API.

Not delivered or not evidenced

- Independently deployed microservices, API gateway, event bus/Kafka/EventBridge, distributed saga and database-per-service.
- Active payment, POS, e-commerce, accounting, shipping and vendor adapters.
- Inter-warehouse stock-transfer workflow, lot/batch/serial traceability, expiry tracking and FIFO/FEFO enforcement.
- Native mobile application; warehouse mobile mode is a browser-based interface/simulation.
- PostgreSQL-backed integration/migration suite and automated frontend/browser test suite.
- Performance/load testing, high-availability deployment, recovery exercise and quantified uptime commitment.

> **Non-functional targets still need evidence:** the development plan calls for API p95 under 200 ms, catalogue search under 500 ms, a 200% traffic increase, 99.95% availability, 60-second recovery time and near-zero recovery-point objectives. The repository's in-memory latency metrics are not a substitute for load, availability or recovery tests.

What was added beyond the documents

The implementation adds engineering/product features beyond the core project processes. These extend the prototype but are not substitutes for unfinished integrations or operational targets.

Additional implementation	How it extends the documented scope
Platform/company and subscription-status administration.	Company-scoped identity is extended into a SaaS-style platform governance workflow.
Company profiles and configurable access/security policies.	Additional administrative configuration beyond the listed staff and warehouse panels.
In-memory per-route request/error counts and latency averages/p95.	Basic observability beyond the core process workflows; it does not establish a measured SLO.
Light/dark theme and a touch-friendly warehouse scanner simulation.	Additional interface/usability capabilities; scanner simulation is not a native mobile application.
Provider-neutral ports for payment, POS, commerce, accounting and shipping.	Integration seams are supplied beyond basic forms, but there are no activated external-system adapters.

AWS-hosted runtime checked on 12 October 2026

Check	Observed result
Worker portal: `http://13.63.238.187/`	HTTP 200; browser displays SyncStock Worker Operations, PostgreSQL Synced and READY.
Admin portal: `http://13.63.238.187/admin.html`	HTTP 200; browser displays SyncStock Admin governance portal.
API health: `/api/health`	HTTP 200; response reported `status: ok` and service `inventory-api`.
API readiness: `/api/health/ready`	HTTP 200; response reported `status: ready` and `storage: postgres`.
Authenticated business workflow	Not exercised. The AWS demo is HTTP-only; credentials or sensitive data should not be entered until HTTPS is enabled.

The implementation roadmap identifies the host as a single AWS EC2 demo in Europe (Stockholm), verified there on 8 October 2026. This review separately checked the deployed pages and live API readiness on 12 October 2026. The single host is a demonstration deployment, not a highly available production service.

Final conclusion

SyncStock 2.0 is a substantial inventory-management prototype aligned with many of the core business workflows documented in the earlier project phases. Its codebase includes a modular TypeScript/Node.js API, React worker and admin portals, and PostgreSQL persistence. It is deployed on a single AWS EC2 instance.

Unlike the earlier unverified local-port check, this review confirmed that the **AWS-hosted worker and admin pages respond** and the live readiness endpoint reports **PostgreSQL storage ready**. The backend test suite also passed 76/76 tests, and both production builds succeeded. These results do not prove every authenticated business flow or transaction end to end; such a workflow was not performed in this review. Nor does the current deployment demonstrate the availability, security or scalability targets in the original proposal.

The principal specified variance is the Phase 4 language: Java is prescribed, whereas the submitted app uses TypeScript/Node.js. It should be presented accurately as an AWS-hosted modular prototype rather than a Java implementation or the fully distributed system envisioned in the design proposals.

Recommended priorities

1. Enable HTTPS on the AWS host before entering any real credentials or business information; secure the hostname, certificate and network configuration.

2. Demonstrate safe, complete authenticated workflows: receiving or counting stock, processing a sale/return, and creating/approving/receiving a PO. Confirm resulting records are persisted to PostgreSQL.

3. Resolve and openly document the Java-versus-TypeScript/Node.js difference against the Phase 4 requirement.

4. Complete the most relevant specified gaps: vendor communication, stock transfer, receiving-condition capture, and database-backed integration tests.

5. Claim production readiness or availability, performance and recovery targets only after HTTPS, security configuration, backups and measured operational tests are verified.

Sources reviewed

- **PROJECT PROBLEM STATEMEN1.docx** - business case, strategic solution and target architecture.
- **Inventory System Development Plan.docx** - workflows and functional/non-functional targets.
- **System Design Specification.docx** - role screens, data model, process and technology design.
- **NPRT63_Phase4_Final_Project_Briefing.pptx** - implementation, prescribed stack, demonstration, testing and report expectations.
- **Implementation Roadmap.md** and **Code/README.md** - delivery status, AWS deployment details and project limitations.
- **Application source, test/build outputs and AWS-hosted runtime checks** - code features, automated verification, live browser pages and API health/readiness.

> **Assessment qualification:** the hosted response and PostgreSQL readiness were directly observed during this review. Authentication and protected transactions remain unverified. The AWS demo is HTTP-

only and must not be used with real passwords or sensitive data.

End of report | Prepared 12 October 2026